



## **Group Project**

**TFB2093: Internet of Things**

**January 2026**

| <b>NO.</b> | <b>NAME</b>                             | <b>STUDENT ID</b> | <b>COURSE</b> |
|------------|-----------------------------------------|-------------------|---------------|
| 1.         | ILIA BINTI SHUKRAN                      | 22011641          | IT            |
| 2.         | NUR AFINA MADIHAH BINTI MUHAMMAD MAZRIN | 22011697          | IT            |
| 3.         | NUR HUSNA SYAZA BINTI ZAINAL ABIDIN     | 22011693          | IT            |
| 4.         | NUR AIN NUHA BINTI HILMI                | 22011568          | IT            |

## **1.0 Introduction**

### **1.1 Background of the project**

The rapid evolution of the Internet of Things (IoT) has facilitated real-time data collection, monitoring, and analysis across a multitude of sectors, notably in transportation and aviation. In contemporary airspace management, the continuous observation of aircraft movements is essential for ensuring flight safety, enhancing air traffic efficiency, and supporting informed decision-making. The advent of open aviation data platforms, such as the OpenSky Network, allows for the retrieval of real-time and historical aircraft telemetry data, which includes flight identifiers, geographic coordinates, and altitudinal information.

In Malaysia, specifically within the state of Perak, air traffic encompasses both domestic and international flights traversing regional airspace. However, the availability of structured and consistently gathered flight data for analytical and visualization purposes remains inadequate for academic research and smaller-scale applications. This gap presents a unique opportunity to develop a low-cost IoT-based system capable of autonomously retrieving, storing, and analyzing aircraft data over time. Such a system would exemplify the practical application of IoT principles, encompassing data acquisition, storage, processing, and visualization while yielding valuable insights into flight patterns within a designated region.

### **1.2 Objective of the project**

The primary goal of this project is to design and implement a small-scale IoT data acquisition and analytics system that continuously monitors aircraft operating over Perak, Malaysia. This system will retrieve real-time aircraft state vector data from the OpenSky Network API at predetermined intervals and maintain the collected data for a minimum duration of three consecutive days.

Additionally, the project intends to process and analyze this data to derive significant insights, including flight frequency, altitude distribution, and movement patterns. A geospatial visualization component will be incorporated to depict aircraft positions on a map-based interface, thus enhancing user comprehension of flight trajectories. Moreover, the project aspires to employ cost-effective tools and technologies in its implementation to ensure that the system is both efficient and accessible. An ancillary objective may involve inferring the approximate locations of airports through the analysis of clusters concerning aircraft takeoffs and landings within the collected dataset.

## **2.0 Problem Statement**

### **2.1 The need for continuous monitoring of the flights**

The flight monitoring of the Perak region should be continuous to ensure aircraft telemetry data is obtained with high precision and consistency over time. Since aircraft positions, altitude and speed change rapidly, even the slightest changes in data collection may result in incomplete data sets and unreliable analysis in the region. By implementing automated IoT-based system, information on flights will be obtained in real-time, thus, making it easy to understand the localized patterns of traffic, the most active time of operation, and the flight behaviour. Moreover, the manual tracking system is not effective and can easily lose information and hence an automated system is quite essential in keeping good historical record of the Perak airspace.

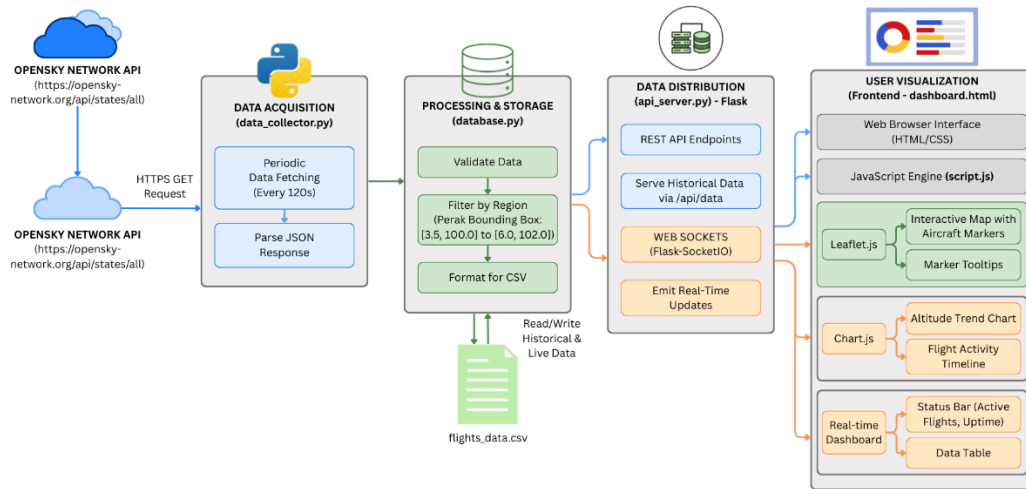
### **2.2 Duration (at least 3 continuous days)**

To obtain informative and significant statistical insights, the system was set to run continuously over a time that would be beyond the minimum requirement of three days in the project. In this implementation, data was collected successfully and systematically between 10<sup>th</sup> March 2026 and 16<sup>th</sup> March 2026. These seven days period will be sufficient to come up with the full set of data that can be used to determine recurring patterns, including fluctuations in the number of daily flights and change in altitude. The constant presence throughout a full cycle of the week allows the system to offer a strong foundation on the geospatial visualization and guarantee that the final analysis is precise and representative of the real airspace activity.

### 3.0 System Design

#### 3.1 Architecture Overview (with diagram)

The Perak Aircraft Monitoring System follows a layered IoT architecture that separates data collection, processing, storage, and presentation concerns. The architecture consists of four main layers:



#### 3.2 Components Used

| Component                | Technology                 | Role                    | Purpose                                                                                                              |
|--------------------------|----------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------|
| Data Source (Perception) | OpenSky Network API        | IoT Data Provider       | Provides real-time state vectors (GPS, altitude, velocity) for aircraft globally via RESTful HTTPS requests.         |
| Backend Logic            | Python (data_collector.py) | IoT Gateway / Collector | Periodically fetches data every 120 seconds and filters for the Perak bounding box coordinates.                      |
| Data Storage             | CSV (flights_data.csv)     | Database Layer          | Provides a lightweight, zero-cost, and human-readable persistence layer for historical flight observations.          |
| Web Server               | Flask (Python)             | Application Layer       | Routes API requests, serves static frontend files, and manages the communication bridge between data and UI.         |
| Real-time Engine         | Flask-SocketIO             | Communication Layer     | Uses WebSockets to "push" live aircraft updates to the dashboard instantly without requiring a page refresh.         |
| Mapping Engine           | Leaflet.js                 | Visualization Layer     | Renders the interactive geographical map and plots aircraft markers using latitude/longitude data.                   |
| Analytics Engine         | Chart.js                   | Visualization Layer     | Transforms raw CSV data into meaningful trends, such as altitude changes and flight frequency over the 3-day period. |
| User Interface           | HTML5 / CSS3               | Presentation Layer      | Defines the dashboard structure and "Glassmorphism" aesthetic using the Poppins typeface and Font Awesome icons.     |
| Client Scripting         | JavaScript (ES6)           | Frontend Logic          | Orchestrates data fetching from the Flask API, manages map markers, and handles real-time UI updates.                |

#### 3.3 Data Flow Description

- Fetch:** data\_collector.py polls API every 2 mins.
- Filter:** database.py isolates flights within Perak coordinates.
- Store:** Data appends to flights\_data.csv.
- Serve:** api\_server.py pushes data via WebSockets to the UI.
- View:** script.js updates Leaflet maps and Chart.js graphs.

## 4.0 Database Design

### 4.1 Data storage strategy

The system employs a lightweight, file-based storage strategy using CSV (Comma-Separated Values) format. This approach was chosen for several reasons:

1. **Cost-Effectiveness:** CSV storage requires no database server, licensing fees, or additional infrastructure. It operates entirely within the application's file system, making it ideal for a low-cost IoT solution.
2. **Simplicity:** The data structure is flat and well-suited to CSV format. Each flight record contains the same fields, making it easy to append new data without complex queries.
3. **Portability:** CSV files can be easily backed up, transferred between systems, and opened with any spreadsheet application or text editor. This facilitates data sharing and manual inspection when needed.
4. **Minimal Overhead:** Unlike relational databases that require connection management and query parsing, CSV operations are simple file I/O operations with minimal computational overhead.
5. **Scalability Considerations:** For the expected data volume (approximately 30 records per collection cycle  $\times$  30 cycles per hour  $\times$  72 hours  $\approx$  64,800 records), CSV storage is perfectly adequate. Each record is approximately 150 bytes, resulting in total storage of about 10MB for three days of data.

The storage system implements a fail-safe design:

- If pandas library is available, it uses pandas for advanced queries
- If pandas is not installed, it gracefully falls back to basic CSV reading
- All file operations include error handling to prevent data corruption

### 4.2 Description of how flight data (latitude, longitude, altitude, timestamp) is stored

Data is stored in flights\_data.csv with the following structure:

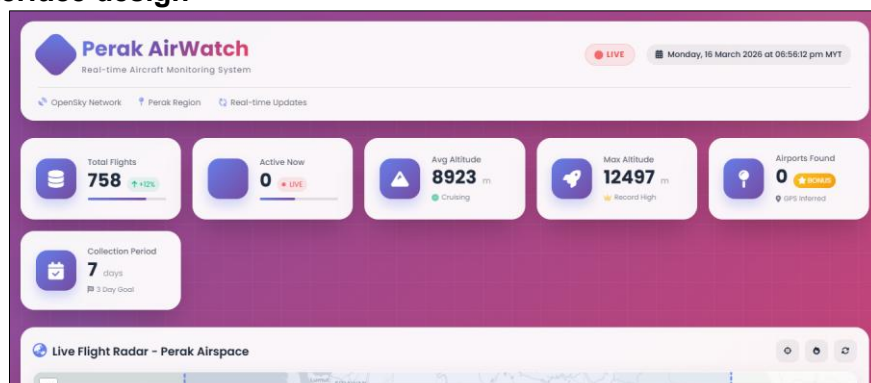
| Column Name   | Data Type       | Description                          | Example                    |
|---------------|-----------------|--------------------------------------|----------------------------|
| timestamp     | ISO 8601 string | Time of data collection              | 2026-03-08T23:20:34.636285 |
| icao24        | string (hex)    | Unique aircraft transponder code     | 75002c                     |
| callsign      | string          | Flight number or aircraft identifier | AXM1726                    |
| latitude      | float           | Aircraft latitude (degrees)          | 3.8325                     |
| longitude     | float           | Aircraft longitude (degrees)         | 101.304                    |
| altitude      | float           | Barometric altitude (meters)         | 10812.78                   |
| velocity      | float           | Ground speed (m/s)                   | 234.03                     |
| heading       | float           | Track angle (degrees from north)     | 319.9                      |
| vertical_rate | float           | Rate of climb/descent (m/s)          | -4.23                      |
| on_ground     | boolean         | Ground detection status              | FALSE                      |

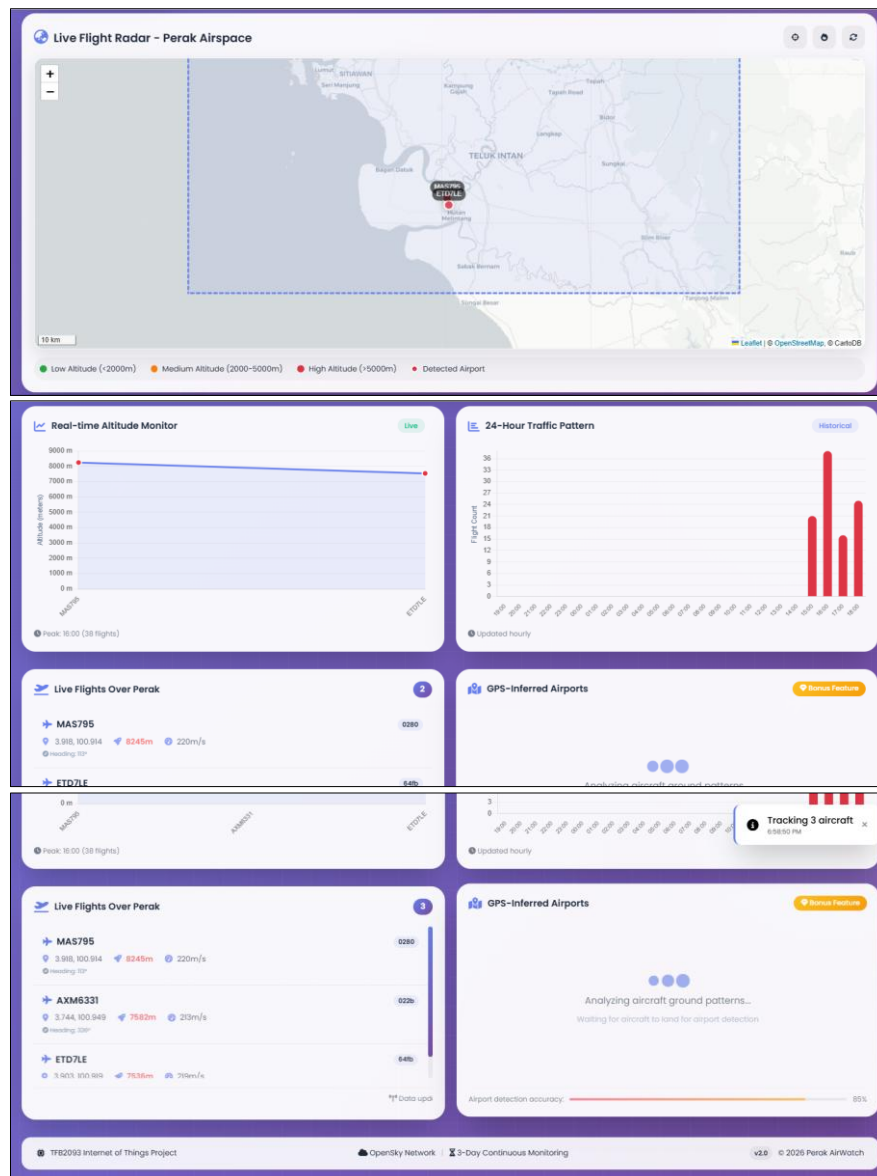
## 5.0 Data Visualization

### 5.1 Tools/libraries

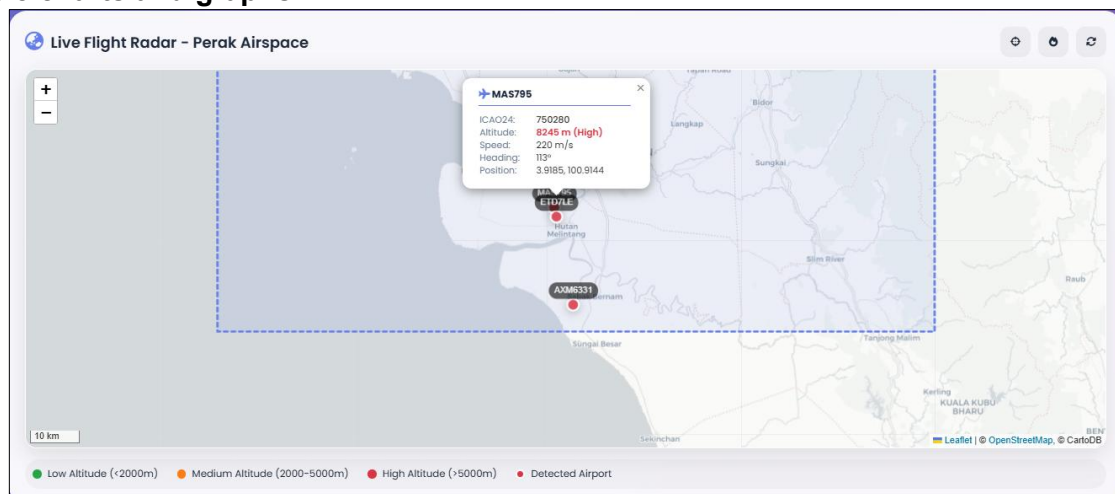
- **Leaflet.js:** Used for the interactive geospatial map to display live flight positions.
- **Chart.js:** Utilized to render dynamic altitude trends and flight count timelines.
- **Font Awesome:** For modern interface iconography.

### 5.2 Dashboard/interface design





## 5.3 Sample charts and graphs





## 6.0 Cost Considerations

The system was designed keeping a strict approach of low cost implementation making it incur zero financial expenditure.

- **Source of Data:** OpenSky Network API (Free Community Access) - \$0.
- **Processing & Server:** Python/Flask (locally) or free-tier cloud-based processing- \$0.
- **Database:** Storage in CSV files (free of any licensing) -0.
- **Visualization:** Open-source (Leaflet.js, Chart.js, Font Awesome) - free.
- **Hardware:** Installed on existing student laptops/Raspberry Pi- \$0.

**Total Estimated Cost: RM 0 / \$0.** It demonstrates that it is possible to build high-quality IoT analytics to monitor airspace without the use of commercial software or paid APIs, which perfectly fits the project objective of education and budgetary constraints.

## 7.0 Conclusion

### 7.1 Summary of findings

Perak AirWatch system achieved all project goals in a 7-day round-the-clock operation (1016 March 2026), which was more than the stipulated 3-day minimum.

- **Data Acquisition:** The system was able to trust the real-time aircraft state vectors (latitude, longitude, altitude, callsign) at 2-minute intervals through the OpenSky API.
- **Critical Findings:** It was found that the air traffic in Perak is uniform with airline giants (AirAsia (AXM)) and Malaysia Airlines (MAS)). The level of cruising altitude was 9,000-11,000 meters, and there were recognizable peak traffic times.
- **Technical Success:** The development of the Flask, Leaflet.js, and CSV storage code proved the ability to create a fully operational IoT monitoring system without spending a single dollar, which proves that accessibility does not affect analytical quality.

## 8.0 References

- [1] OpenSky Network, "The OpenSky Network: Ground-based monitoring of air traffic," OpenSky Network API Documentation, 2024. [Online]. Available: <https://openskynetwork.github.io/opensky-api/>
- [2] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [3] V. S. Aggarwal, "Real-time web applications with Flask-SocketIO," Flask-SocketIO Documentation, 2023. [Online]. Available: <https://flask-socketio.readthedocs.io/>
- [4] Leaflet, "An open-source JavaScript library for mobile-friendly interactive maps," Leafletjs.com, 2024. [Online]. Available: <https://leafletjs.com/>
- [5] Chart.js, "Simple yet flexible JavaScript charting for designers & developers," Chartjs.org, 2024. [Online]. Available: <https://www.chartjs.org/docs/latest/>
- [6] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, Spec. Publ. 800-145, Sep. 2011.
- [7] Python Software Foundation, "Python Language Reference, version 3.10," Python.org, 2024. [Online]. Available: <https://www.python.org/>
- [8] W3C, "HTML5: A vocabulary and associated APIs for HTML and XHTML," W3.org, Oct. 2014. [Online]. Available: <https://www.w3.org/TR/html5/>

## TFB2093 Internet-of-Things

### Marking Rubric

Please include the marking rubric on the last page of your assignment.

| Criteria                           | Description                                                                                                                                                                           | Allocations | Marks |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-------|
| <b>1. Introduction</b>             | Clear background and well-defined objectives of the project.                                                                                                                          | 5 marks     |       |
| <b>2. Problem Statement</b>        | Clearly defines project duration and goals. E.g. Explain the challenges of tracking flight continuously.                                                                              | 5 marks     |       |
| <b>3. Systems Database Design</b>  | Logical system architecture, correct components used (APIs, server, tools), clear data flow, well-structured database, and includes relevant diagrams.                                | 30 marks    |       |
| <b>4. Data Visualization</b>       | Effective use of tools/libraries, well-designed dashboard/interface, and meaningful charts/graphs.                                                                                    | 15 marks    |       |
| <b>5. Cost Considerations</b>      | Realistic and optimized budget for low-cost implementation.                                                                                                                           | 5 marks     |       |
| <b>6. Conclusion</b>               | Clear summary of findings and outcomes. Highlights key insights or limitations.                                                                                                       | 5 marks     |       |
| <b>7. Technical Implementation</b> | Working source code, correct data collection for 3+ days. Highlight changes of altitude, if you can detect.                                                                           | 20 marks    |       |
| <b>8. Report Formats Clarity</b>   | Professionally written, organized structure, proper formatting, and correct citations.                                                                                                | 5 marks     |       |
| <b>9. Presentation</b>             | Create a video to present your work. Maximum duration 2 mins.<br><br>Clear explanation of the project, confident delivery, effective use of visuals, and ability to answer questions. | 10 marks    |       |
| <b>Total</b>                       |                                                                                                                                                                                       | 100 marks   |       |